Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №9
Сетевое взаимодействие в Python: TCP/UDP и клиент-серверная архитектура

1. Цель работы

Получить практические навыки разработки сетевых приложений на Python и понять принципы клиент-серверного взаимодействия. В ходе работы необходимо изучить понятия IP-адреса, порта, сокета, протоколов TCP и UDP, освоить модуль socket, реализовать TCP-сервер и TCP-клиент, разработать собственный прикладной протокол на основе JSON, организовать обработку нескольких клиентов, а также обработать разрывы соединения, тайм-ауты и ошибки.

2. Постановка задачи

Разработать клиент-серверное приложение для предметной области «Учебные задачи» (Вариант 5). Основная сущность — Task со следующими полями: id, title (название), priority (приоритет), status (статус), assignee (исполнитель). Специализированная команда варианта — filter_by_priority, получение задач по приоритету.

Требования к приложению:

• клиент и сервер размещаются в разных файлах, бизнес-логика отделена от сетевого кода, используются Repository и Service;

• взаимодействие по TCP через модуль socket, сервер обслуживает несколько клиентов одновременно;

• собственный прикладной протокол на основе JSON с разделителем \n, не менее шести команд и единый формат ответа;

• обработка ошибок: недоступный сервер, разрыв соединения, неизвестная команда, некорректный JSON, отсутствующие поля, ненайденный объект, неверные значения, тайм-аут;

• тайм-ауты на клиенте и на сервере, логирование через модуль logging;

• не менее пяти автоматических тестов (репозиторий, сервис, протокол);

3. Краткие теоретические сведения

Клиент-серверная архитектура предполагает два взаимодействующих процесса: клиент инициирует соединение и отправляет запросы, сервер ожидает подключения и обрабатывает запросы. Клиентом может быть консольная программа, браузер, мобильное приложение или другой сервер.

IP-адрес определяет узел сети, а порт — конкретное приложение на этом узле. Например, 127.0.0.1:5000 означает приложение, ожидающее соединения на текущем компьютере (localhost) на порту 5000. Сокет — программный интерфейс для обмена данными через сеть; в Python он создаётся функцией socket.socket().

TCP — протокол с установлением соединения: данные доставляются в правильном порядке, потери повторяются, приложение получает непрерывный поток байтов. UDP соединения не устанавливает: сообщения передаются отдельными датаграммами, доставка и порядок не гарантируются, зато накладные расходы ниже. Сравнение протоколов приведено в таблице 9.1.

Свойство	**TCP**	**UDP**
Соединение	устанавливается	не устанавливается
Доставка	гарантируется	не гарантируется
Порядок данных	сохраняется	не сохраняется
Единица передачи	поток байтов	датаграмма
Типичное применение	команды, файлы, запросы	статусы, телеметрия, игры

Важная особенность TCP: один вызов send() не соответствует одному вызову recv(), поскольку TCP передаёт поток байтов без границ сообщений. Поэтому прикладной протокол сам определяет, где заканчивается одно сообщение: в данной работе каждое JSON-сообщение завершается символом \n (newline-delimited JSON).

Прикладной протокол — набор правил о формате и последовательности сообщений. JSON удобен для обмена структурированными данными: он текстовый, читаемый и поддерживается стандартной библиотекой Python.

4. Архитектура приложения

Приложение разделено на слои: сетевой уровень принимает соединения и разбирает сообщения, сервисный слой содержит бизнес-логику и проверки, уровень хранения отвечает за данные. Схема взаимодействия приведена на рисунке 9.1.

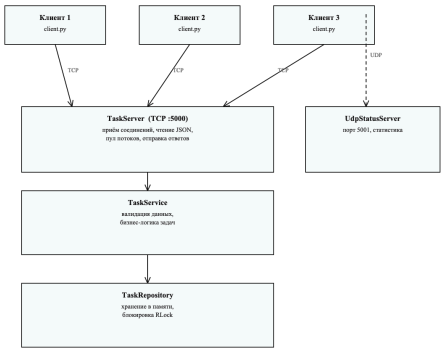


Рисунок 9.1 – Архитектура клиент-серверного приложения

Модуль	**Назначение**
models.py	сущность Task и перечисления Priority, TaskStatus
repository.py	хранение задач в памяти, потокобезопасные операции
service.py	бизнес-логика: создание, поиск, изменение, удаление, filter_by_priority
protocol.py	кодирование и декодирование JSON-сообщений, границы сообщений
server.py	TCP-сокет, пул потоков, диспетчеризация команд
client.py	подключение, меню, отправка запросов, вывод ответов
exceptions.py	собственные исключения ValidationError, NotFoundError, ProtocolError

5. Протокол прикладного уровня

Запрос содержит поля command и data, ответ — status и data (при успехе) либо status и error (при ошибке). Каждое сообщение — один JSON-объект, завершающийся символом \n.

```
{"command": "get", "data": {"id": 2}}
{"status": "ok", "data": {"task": {...}}}
{"status": "error", "error": "Задача с ID 999 не найдена"}
```

Команды протокола приведены в таблице 9.2.

Команда	Запрос	Ответ	Назначение
list	{"command": "list", "data": {}}	{"status": "ok", "data": {"tasks": [], "count": 0}}	список задач
get	{"command": "get", "data": {"id": 2}}	{"status": "ok", "data": {"task": {...}}}	одна задача
create	{"command": "create", "data": {"title": "...", "priority": "высокий", "assignee": "..."}}	{"status": "ok", "data": {"task": {...}}}	создание
update	{"command": "update", "data": {"id": 2, "status": "в работе"}}	{"status": "ok", "data": {"task": {...}}}	изменение
delete	{"command": "delete", "data": {"id": 3}}	{"status": "ok", "data": {"deleted": 3}}	удаление
filter_by_priority	{"command": "filter_by_priority", "data": {"priority": "высокий"}}	{"status": "ok", "data": {"tasks": [], "count": 0}}	задачи по приоритету

6. Реализация

6.1. Модель данных

Файл: models.py

```
from dataclasses import dataclass
from enum import Enum
from typing import Any, Dict


class Priority(Enum):
    LOW = "низкий"
    MEDIUM = "средний"
    HIGH = "высокий"


class TaskStatus(Enum):
    NEW = "новая"
    IN_PROGRESS = "в работе"
    DONE = "выполнена"


# Основная сущность предметной области «учебная задача»:
# идентификатор и четыре дополнительных поля.
@dataclass
```

```
class Task:
    id: int
    title: str
    priority: Priority
    status: TaskStatus
    assignee: str

    def to_dict(self) -> Dict[str, Any]:
        return {
            "id": self.id,
            "title": self.title,
            "priority": self.priority.value,
            "status": self.status.value,
            "assignee": self.assignee,
        }

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> "Task":
        return cls(
            id=int(data["id"]),
            title=str(data["title"]),
            priority=Priority(str(data["priority"])),
            status=TaskStatus(str(data["status"])),
            assignee=str(data["assignee"]),
        )
```

6.2. Репозиторий

Несколько клиентских потоков обращаются к хранилищу одновременно, поэтому все операции защищены блокировкой RLock — это исключает состояние гонки при параллельных запросах.

Файл: repository.py

```
from threading import RLock
from typing import Dict, List, Optional

from models import Task


# Хранилище задач в памяти процесса. Несколько клиентских потоков
# обращаются к нему одновременно, поэтому операции защищены блокировкой.
class TaskRepository:
    def __init__(self):
        self._tasks: Dict[int, Task] = {}
        self._lock = RLock()

    def add(self, task: Task) -> None:
        with self._lock:
            if task.id in self._tasks:
                raise ValueError(f"Задача с ID {task.id} уже существует")
            self._tasks[task.id] = task

    def get(self, task_id: int) -> Optional[Task]:
        with self._lock:
            return self._tasks.get(task_id)

    def get_all(self) -> List[Task]:
        with self._lock:
            return list(self._tasks.values())

    def update(self, task: Task) -> None:
        with self._lock:
            if task.id not in self._tasks:
                raise ValueError(f"Задача с ID {task.id} не найдена")
            self._tasks[task.id] = task

    def delete(self, task_id: int) -> bool:
```

```
        with self._lock:
            return self._tasks.pop(task_id, None) is not None

    def next_id(self) -> int:
        with self._lock:
            return max(self._tasks, default=0) + 1
```

6.3. Сервис

Сервис выполняет проверки и не знает ни про сокеты, ни про JSON. Некорректные данные приводят к исключению ValidationError, отсутствующая задача — к NotFoundError.

Файл: service.py

```
from typing import Any, Dict, List, Optional

from exceptions import NotFoundError, ValidationError
from models import Priority, Task, TaskStatus
from repository import TaskRepository


# Бизнес-логика работы с учебными задачами.
# Сервис не знает ни про сокеты, ни про JSON: он работает с объектами Task.
class TaskService:
    def __init__(self, repository: TaskRepository):
        self._repository = repository

    def list_tasks(self) -> List[Task]:
        return self._repository.get_all()

    def get_task(self, task_id: int) -> Task:
        task = self._repository.get(task_id)
        if task is None:
            raise NotFoundError(f"Задача с ID {task_id} не найдена")
        return task

    def create_task(self, title: str, priority: str, assignee: str) -> Task:
        task = Task(
            id=self._repository.next_id(),
            title=self._validate_text(title, "Название задачи"),
            priority=self._parse_priority(priority),
            status=TaskStatus.NEW,
            assignee=self._validate_text(assignee, "Исполнитель"),
        )
        self._repository.add(task)
        return task

    def update_task(self, task_id: int, changes: Dict[str, Any]) -> Task:
        task = self.get_task(task_id)

        if "title" in changes:
            task.title = self._validate_text(changes["title"], "Название
задачи")
        if "priority" in changes:
            task.priority = self._parse_priority(changes["priority"])
        if "status" in changes:
            task.status = self._parse_status(changes["status"])
        if "assignee" in changes:
            task.assignee = self._validate_text(changes["assignee"],
"Исполнитель")

        self._repository.update(task)
        return task

    def delete_task(self, task_id: int) -> None:
        if not self._repository.delete(task_id):
            raise NotFoundError(f"Задача с ID {task_id} не найдена")
```

```
    # Специализированная операция варианта: получение задач по приоритету
    def filter_by_priority(self, priority: str) -> List[Task]:
        value = self._parse_priority(priority)
        return [task for task in self._repository.get_all() if task.priority ==
value]

    # Статистика используется UDP-операцией «статус сервера»
    def get_statistics(self) -> Dict[str, Any]:
        tasks = self._repository.get_all()
        by_status = {status.value: 0 for status in TaskStatus}
        for task in tasks:
```

6.4. Протокол

Файл: protocol.py

```
import json
from typing import Any, Dict, Optional

from exceptions import ProtocolError

DELIMITER = b"\n"
MAX_MESSAGE_SIZE = 64 * 1024


# Прикладной протокол: newline-delimited JSON.
# Каждое сообщение — один JSON-объект, завершающийся символом \n,
# поэтому граница сообщения определяется однозначно.
def encode_message(payload: Dict[str, Any]) -> bytes:
    try:
        text = json.dumps(payload, ensure_ascii=False)
    except (TypeError, ValueError) as error:
        raise ProtocolError(f"Сообщение невозможно сериализовать: {error}")
    return (text + "\n").encode("utf-8")


def _decode_object(raw: bytes) -> Dict[str, Any]:
    if len(raw) > MAX_MESSAGE_SIZE:
        raise ProtocolError("Сообщение превышает допустимый размер")
    try:
        text = raw.decode("utf-8")
    except UnicodeDecodeError:
        raise ProtocolError("Сообщение не является корректным UTF-8")

    text = text.strip()
    if not text:
        raise ProtocolError("Пустое сообщение")

    try:
        payload = json.loads(text)
    except json.JSONDecodeError as error:
        raise ProtocolError(f"Некорректный JSON: {error.msg}")

    if not isinstance(payload, dict):
        raise ProtocolError("Сообщение должно быть JSON-объектом")
    return payload


def decode_message(raw: bytes) -> Dict[str, Any]:
    payload = _decode_object(raw)
    if "command" not in payload:
        raise ProtocolError("В запросе отсутствует поле command")
    if not isinstance(payload["command"], str):
        raise ProtocolError("Поле command должно быть строкой")

    payload.setdefault("data", {})
    if not isinstance(payload["data"], dict):
        raise ProtocolError("Поле data должно быть JSON-объектом")
    return payload


def decode_response(raw: bytes) -> Dict[str, Any]:
```

```
    payload = _decode_object(raw)
    if payload.get("status") not in ("ok", "error"):
        raise ProtocolError("Некорректное поле status в ответе")
    if payload["status"] == "error" and "error" not in payload:
        raise ProtocolError("В ответе об ошибке отсутствует поле error")
    payload.setdefault("data", {})
    return payload


def make_request(command: str, data: Optional[Dict[str, Any]] = None) ->
Dict[str, Any]:
    return {"command": command, "data": data or {}}


def make_ok(data: Optional[Dict[str, Any]] = None) -> Dict[str, Any]:
    return {"status": "ok", "data": data or {}}


def make_error(message: str) -> Dict[str, Any]:
    return {"status": "error", "error": message}
```

6.5. Сервер

Сервер принимает соединения в основном потоке, а каждую сессию передаёт пулу потоков ThreadPoolExecutor. Буфер накапливает байты и выделяет сообщения по символу \n, поэтому границы сообщений не зависят от разбиения на пакеты.

Файл: server.py

```
        self._host = host
        self._port = port
        self._max_workers = max_workers
        self._socket: Optional[socket.socket] = None
        self._executor = ThreadPoolExecutor(max_workers=max_workers)
        self._stopped = threading.Event()

    def start(self) -> int:
        self._socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self._socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self._socket.bind((self._host, self._port))
        self._socket.listen(MAX_WORKERS)
        self._socket.settimeout(0.5)
        self._host, self._port = self._socket.getsockname()
        log.info("Server started on %s:%s", self._host, self._port)
        return self._port

    def serve_forever(self) -> None:
        if self._socket is None:
            self.start()
        try:
            while not self._stopped.is_set():
                try:
                    connection, address = self._socket.accept()
                except socket.timeout:
                    continue
                except OSError:
                    break
                self._executor.submit(self._handle_client, connection, address)
        finally:
            self._executor.shutdown(wait=False)
            if self._socket is not None:
                self._socket.close()
            log.info("Server stopped")

    def stop(self) -> None:
        self._stopped.set()
```

```
        if self._socket is not None:
            try:
                self._socket.close()
            except OSError:
                pass

    def _handle_client(self, connection: socket.socket, address) -> None:
        client = f"{address[0]}:{address[1]}"
        log.info("Client connected: %s", client)
        connection.settimeout(CLIENT_TIMEOUT)
        buffer = b""
        try:
            with connection:
                while not self._stopped.is_set():
                    try:
                        chunk = connection.recv(4096)
                    except socket.timeout:
                        log.warning("Client timeout: %s", client)
                        break
                    except ConnectionResetError:
                        log.warning("Connection reset by client: %s", client)
                        break
                    if not chunk:
                        break
                    buffer += chunk
                    while b"\n" in buffer:
                        line, buffer = buffer.split(b"\n", 1)
                        if not line.strip():
                            continue
                        response = self._process_request(line, client)
                        connection.sendall(encode_message(response))
        except OSError as error:
            log.warning("Connection error with %s: %s", client, error)
        finally:
            log.info("Client disconnected: %s", client)

    def _process_request(self, raw: bytes, client: str) -> Dict[str, Any]:
        try:
            request = decode_message(raw)
            log.info("Command: %s from %s", request["command"], client)
            data = self._dispatch(request)
            log.info("Command %s completed", request["command"])
            return make_ok(data)
        except (ProtocolError, ValidationError, NotFoundError) as error:
            log.warning("Bad request from %s: %s", client, error)
            return make_error(str(error))
        except Exception as error:
            log.exception("Unhandled error: %s", error)
            return make_error("Внутренняя ошибка сервера")

    def _dispatch(self, request: Dict[str, Any]) -> Dict[str, Any]:
        command = request["command"]
        data = request["data"]
        handlers = {
            "list": self._list,
            "get": self._get,
            "create": self._create,
            "update": self._update,
            "delete": self._delete,
            "filter_by_priority": self._filter_by_priority,
        }
        handler = handlers.get(command)
        if handler is None:
            raise ProtocolError(f"Неизвестная команда: {command}")
        return handler(data)

    def _list(self, data: Dict[str, Any]) -> Dict[str, Any]:
        tasks = self._service.list_tasks()
        return {"tasks": [task.to_dict() for task in tasks], "count":
len(tasks)}
```

Диспетчеризация команд выполнена словарём обработчиков: добавление новой команды не требует изменения сетевого кода.

Файл: server.py (фрагмент диспетчеризации)

```
    def _get(self, data: Dict[str, Any]) -> Dict[str, Any]:
        task = self._service.get_task(self._task_id(data))
        return {"task": task.to_dict()}

    def _create(self, data: Dict[str, Any]) -> Dict[str, Any]:
        task = self._service.create_task(
            title=data.get("title"),
            priority=data.get("priority"),
            assignee=data.get("assignee"),
        )
        return {"task": task.to_dict()}

    def _update(self, data: Dict[str, Any]) -> Dict[str, Any]:
        task_id = self._task_id(data)
        changes = {key: value for key, value in data.items() if key != "id"}
        if not changes:
            raise ValidationError("Не переданы поля для изменения")
        task = self._service.update_task(task_id, changes)
        return {"task": task.to_dict()}

    def _delete(self, data: Dict[str, Any]) -> Dict[str, Any]:
        task_id = self._task_id(data)
```

6.6. Клиент

Файл: client.py

```
import json
import socket
import sys
from typing import Any, Dict, Optional

from protocol import decode_response, encode_message, make_request
from server import HOST, TCP_PORT, UDP_PORT

CLIENT_TIMEOUT = 5
MAX_RESPONSE_SIZE = 64 * 1024


class TaskClient:
    def __init__(self, host: str = HOST, port: int = TCP_PORT, timeout: float =
CLIENT_TIMEOUT):
        self._host = host
        self._port = port
        self._timeout = timeout
        self._socket: Optional[socket.socket] = None
        self._buffer = b""

    def connect(self) -> None:
        self._socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self._socket.settimeout(self._timeout)
        self._socket.connect((self._host, self._port))

    def close(self) -> None:
        if self._socket is not None:
            self._socket.close()
            self._socket = None

    # Отправка запроса и получение одного ответа (до символа \n)
    def send_request(self, command: str, data: Optional[Dict[str, Any]] = None)
-> Dict[str, Any]:
        if self._socket is None:
            raise ConnectionError("Соединение с сервером не установлено")
        self._socket.sendall(encode_message(make_request(command, data)))
        return self._receive_response()
```

```
    def _receive_response(self) -> Dict[str, Any]:
        while b"\n" not in self._buffer:
            chunk = self._socket.recv(4096)
            if not chunk:
                raise ConnectionError("Соединение разорвано сервером")
            self._buffer += chunk
            if len(self._buffer) > MAX_RESPONSE_SIZE:
                raise ValueError("Ответ сервера слишком большой")
        line, self._buffer = self._buffer.split(b"\n", 1)
        return decode_response(line)


def udp_status(host: str = HOST, port: int = UDP_PORT) -> Dict[str, Any]:
    udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    udp_socket.settimeout(CLIENT_TIMEOUT)
    try:
        udp_socket.sendto(encode_message(make_request("status")), (host, port))
        raw, _ = udp_socket.recvfrom(4096)
        return json.loads(raw.decode("utf-8"))
    finally:
        udp_socket.close()
```

Меню клиента содержит шесть команд протокола. При недоступном сервере выводится сообщение об ошибке подключения, при отсутствии ответа срабатывает тайм-аут, при разрыве соединения цикл завершается.

6.7. Обработка ошибок и тайм-ауты

Файл: exceptions.py

```
class TaskServiceError(Exception):
    """Базовое исключение предметной области."""


class ValidationError(TaskServiceError):
    """Переданы некорректные данные."""


class NotFoundError(TaskServiceError):
    """Задача не найдена."""


class ProtocolError(Exception):
    """Нарушен формат сообщения прикладного протокола."""
```

Клиент использует тайм-аут 5 секунд на подключение и ожидание ответа, сервер — 30 секунд на сессию. Тайм-ауты не позволяют программе зависнуть навсегда, если узел недоступен или не отвечает. При ошибке протокола сервер возвращает стандартизированный ответ status=error и продолжает обслуживание клиента; при разрыве соединения сессия закрывается, а сервер продолжает принимать новых клиентов.

6.8. Логирование

Сервер пишет в консоль и в файл server.log дату, уровень и сообщение: запуск сервера, подключение и отключение клиента, полученную команду, результат выполнения, ошибочные запросы и исключения.

7. Тестирование

Написано 19 автоматических тестов: четыре теста репозитория, пять тестов сервиса, шесть тестов протокола (включая проверку тайм-аута) и четыре интеграционных теста, которые поднимают реальный TCP-сервер на свободном порту и проходят полный путь «клиент — TCP — сервер — сервис — репозиторий». Результат выполнения приведён на рисунке 9.2.

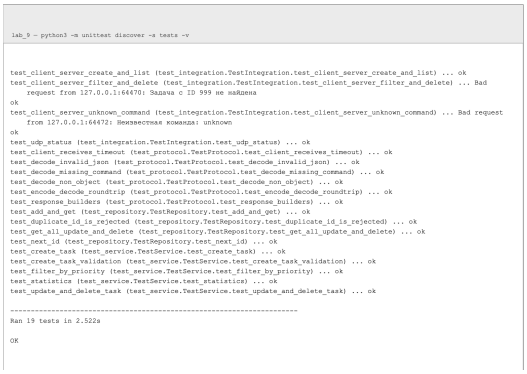


Рисунок 9.2 – Результаты автоматических тестов

8. Демонстрация работы

Тест 1. Запуск сервера и логирование подключений (рисунок 9.3). В журнал записываются запуск сервера, подключение клиента, полученные команды и отключение.

```
lab_9 — python3 server.py


2026-10-18 13:21:27 INFO UDP status server started on 127.0.0.1:5001
2026-10-18 13:21:27 INFO Server started on 127.0.0.1:5000
Сервер запущен на 127.0.0.1:5000, UDP-статус на 5001
2026-10-18 13:21:29 INFO Client connected: 127.0.0.1:64446
2026-10-18 13:21:29 INFO Client connected: 127.0.0.1:64445
2026-10-18 13:21:29 INFO Client connected: 127.0.0.1:64447
2026-10-18 13:21:30 INFO Command: list from 127.0.0.1:64446
2026-10-18 13:21:30 INFO Command list completed
2026-10-18 13:21:31 INFO Command: get from 127.0.0.1:64445
2026-10-18 13:21:31 INFO Command get completed
2026-10-18 13:21:31 INFO Command: filter_by_priority from 127.0.0.1:64447
2026-10-18 13:21:31 INFO Command filter_by_priority completed
2026-10-18 13:21:32 INFO Command: create from 127.0.0.1:64446
2026-10-18 13:21:32 INFO Command create completed
2026-10-18 13:21:33 INFO Command: update from 127.0.0.1:64445
2026-10-18 13:21:33 INFO Command update completed
2026-10-18 13:21:33 INFO Command: create from 127.0.0.1:64447
2026-10-18 13:21:33 INFO Command create completed
2026-10-18 13:21:34 INFO Command: filter_by_priority from 127.0.0.1:64446
2026-10-18 13:21:34 INFO Command filter_by_priority completed
2026-10-18 13:21:34 INFO Client disconnected: 127.0.0.1:64447
2026-10-18 13:21:34 INFO Command: list from 127.0.0.1:64445
```

Рисунок 9.3 – Запуск сервера и журнал событий

Тест 2. Работа клиента: получение списка задач, создание новой задачи и фильтрация по приоритету (рисунок 9.4).

```
lab_9 — python3 client.py (клиент 1)


=== КЛИЕНТ УЧЕБНЫХ ЗАДАЧ ===
1. Получить список задач
2. Получить задачу по ID
3. Создать задачу
4. Изменить задачу
5. Удалить задачу
6. Задачи по приоритету
7. Статус сервера (UDP)
0. Выход
Введите команду: 1
   Елисеева Е. Р.       Подготовить отчёт по лабораторной работе
 ID  Приоритет  Статус      Исполнитель          Название
  1  средний    новая       Протокол Т. Т.
  2  средний    новая       Иванов И. И.
  3  высокий    новая       Петров П. П.         Оформить список
Введите команду: 3
Всего задач: 3

Название: Сдать лабораторную работу №9
   высокий    новая       Елисеева Е. Р.       Сдать лабораторную работу №9
Исполнитель: Елисеева Е. Р.
Приоритет (низкий/средний/высокий):
 ID  Название                  Статус      Исполнитель          Название
Протокол Т. Т.
Иванов И. И.
Петров П. П.
Елисеева Е. Р.
Елисеева Е. Р.
Елисеева Е. Р
```

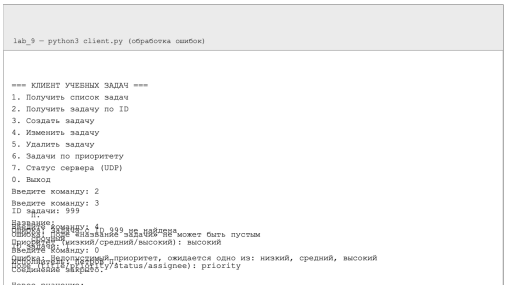


Рисунок 9.5 – Обработка ошибочных запросов

Тест 4. Три клиента работают одновременно: каждый выполняет свои команды, сервер обслуживает сессии параллельно (рисунок 9.6).

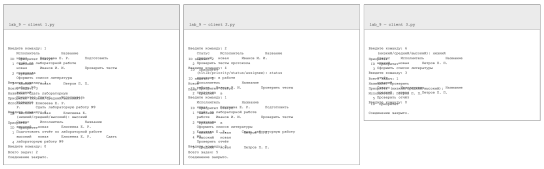


Рисунок 9.6 – Три одновременно работающих клиента

9. Вывод

В ходе лабораторной работы было разработано клиент-серверное приложение для предметной области «Учебные задачи»: TCP-сервер на модуле socket, консольный клиент, собственный JSON-протокол с разделителем \n и шесть команд, включая специализированную filter_by_priority. Сервер обслуживает несколько клиентов одновременно через пул потоков, ведёт журнал событий и корректно обрабатывает разрывы соединения, некорректный JSON и ошибочные данные.

Разделение на сетевой, сервисный и инфраструктурный слои позволило протестировать бизнес-логику отдельно от сети и дополнить проверки

интеграционными тестами, которые проходят полный путь от клиента до репозитория.